

LINIE awk 'FS+=#(NF-1);END{print \$1}'
SUMA TOATE NRELE awk 'for(i=1;NF;i++) s=s+\$1;END{print s}'
FRECVENTA CUVINTELE awk 'for(i=1;NF;i++) {a[\$1]++;} END{for(i in a) print i, a[i]}'
LINIE CU +SC +NR LINIE awk 'NR%5==0{print NR}'

Friday, July 3, 2026 2:06 PM

GREP/SED/AWK LINII DE LINGUIE IMPAR, ÎNCEP

GREP Syntax: grep [options] [pattern] [file]
 -x: POTRIVIRE LINIE
 -E: INTERPRETS +, ? etc AS FUNCTIONS
 -i: CASE INSENSITIVE MATCHING
 -v: INVERT NERE (NON-MATCHING)
 -c: COUNT
 -n: SHOW LINE NUMBERS
 -w: WHOLE WORDS ONLY
 -o: PRINT THE MATCHED PART, NOT THE LINE

NETCARACTERE - orice caracter
 - \: Escapare
 - []: Interval
 - [^]: NEGATIVE
 - ? : 0 SAU 1 APARIȚIE
 - * : GRUPARE (sau) - STRUCTURA
 - { } : GRUPARE (sau) - STRUCTURA
 - [a-z]: alfabetul mic
 - [A-Z]: alfabetul mare
 - [0-9]: cifre
 - [[:alpha:]]: alfabetul
 - [[:digit:]]: cifre

CLASE DE CARACTERE
 [[:alpha:]] litera mică sau mare
 [[:digit:]] 0-9
 [[:upper:]] alfabetul mare
 [[:lower:]] alfabetul mic
 [[:space:]] spațiu
 [[:punct:]] punctuație
 [[:alnum:]] alfanumeric
 [[:print:]] imprimabil
 [[:cntrl:]] control

SED sed [options] [command]
 -e: comandă
 -f: fișier cu comenzi
 -i: editare în loc
 -s: separator simbol linie
 -r: regulă de expresie regulată
 -u: unitate de măsură
 -v: inversare
 -w: cuvânt întreg
 -x: linie întreagă
 -y: linie întreagă
 -z: linie întreagă
 -Z: linie întreagă
 -E: expresie regulată
 -F: separator simbol linie
 -G: grupare
 -H: linie întreagă
 -I: linie întreagă
 -J: linie întreagă
 -K: linie întreagă
 -L: linie întreagă
 -M: linie întreagă
 -N: linie întreagă
 -O: linie întreagă
 -P: linie întreagă
 -Q: linie întreagă
 -R: linie întreagă
 -S: linie întreagă
 -T: linie întreagă
 -U: linie întreagă
 -V: linie întreagă
 -W: linie întreagă
 -X: linie întreagă
 -Y: linie întreagă
 -Z: linie întreagă

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

SED -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'
 -E 's/[a-z][0-9]*/[a-z][0-9]*/'

PROCESE
 sem -init(&sem, 0, 1); // 1 = la
 fork() + wait + exit
 pid = fork();
 if (pid == 0) {
 // child
 // ...
 exit(0);
 }
 else {
 // parent
 wait(&status);
 }

FIFO UN PROCES SCRIE
 ACTUL CITEȘTE
 char *nume_fifo = "/tmp/Alfa.5m";
 mkfifo(nume_fifo, 0666);
 pid = fork();
 if (pid == 0) {
 // child
 // ...
 exit(0);
 }
 else {
 // parent
 // ...
 wait(&status);
 }

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

THREADS
 pthread_t t1, t2;
 pthread_create(&t1, NULL, func, (void*)1);
 pthread_create(&t2, NULL, func, (void*)2);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);

PROCESS STATES TOMPEA MARIA-DARIA
 New - proces de creare, nu este încă în tabela de
 Ready - procesul e pregătit să ruleze,
 asteapta doar CPU-ul
 Running - procesul
 rulează efectiv pe CPU/core
 Waiting/
 Blocked - procesul așteaptă un eveniment/
 și nu poate rula chiar dacă are CPU disponibil
 Terminated (Zombie) - procesul s-a încheiat
 dar părințele nu a făcut wait.
 Diagrama de
 Trazitii

READY - (scheduler dispatch : i se dă CPU) - **READY**
RUNNING - (timeout/preemptie/scheduler ii ia CPU-ul) - **READY**
RUNNING - (core I/O sau așteaptă event/lock) - **WAITING**

WAITING - (evenimentul s-a produs/I/O s-a terminat) - **READY**
RUNNING - (exit()) - **TERMINATED** (nu există tranziție
 directă WAITING -> RUNNING CÂND SE SCHIMBĂ STAREA DIN
 RUN în READY? - când planificatorul îi reține CPU-ul de
 procesul e în continuare gata să ruleze (nu e blocat) - de
 obicei la expirarea cuantelor de timp, sau la apariția unui proces
 cu prioritate mai mare. RUN -> WAITING? Când procesul
 face un apel de sistem care necesită așteptare: read() de
 pe disc, wait() pentru un copil, sem_wait() semnal = 0 etc.
 DE CE PROCESELE ZOMBIE SUNT PROBLEMATICE? Ocupă un slot
 în tabela de procese dar nu consumă CPU/memorie ->
 PID-uri indisponibile. CRITERII DE PLANIFICARE

FIFO (First Come First Served) - se execută în ordine
 sosirii, fără întreruperi; simplu dar poate avea CONVOY EFFECT
 (un job lung blochează multe scurte) și F (Shortest Job First)
 - aleg la momentul în care CPU e liber, jobul cu cea mai
 scurtă durată TOTALă, optim pentru waiting time mediu dacă
 toate joburile sosesc simultan - ALGORITMI NON PREEMPTIVI

ALGORITMI PREEMPTIVI STCF/SRTF (Shortest Time-to-Completion
 First/ Shortest remaining time first) - varianta preemptivă
 a SJF - la sosirea unui job nou, dacă timpul lui rămas e
 mai mic decât timpul rămas al jobului curent, planificatorul
 întrerupe jobul curent - minimizarea optim waiting time
 mediu când joburile sosesc la momente diferite.

ROUND ROBIN (RR) - fiecare proces primește un CUAINTA de timp
 fixă, dacă nu termină în acel timp, e pus la finalul cozii READY
 și următorul e al lui; cuanta mică - răspuns rapid dar overhead
 mare de context switch; cuanta mare -> apropiere de FCFS

PRIORITY SCHEDULING - rulează procesul cu cea mai mare prioritate
 - problema STARVATION (proces cu prioritate mică nu rulează niciodată)
 - soluție AGING (prioritatea crește progresiv cu timpul de așteptare)
 MLFB (Multi Level Feedback) - mai multe cozi, cu priorități
 dif. și cuante diferite (cozi de pr. 1 = cuanta, și invers)

- un proces care consumă toată cuanta e retrogradat la coada
 de pr. mai mică; (un proces care are I/O rapid rămâne promo-
 vat la coada de prioritate mai mare) EDF (Earliest Deadline
 First) - rulează mereu jobul cu cel mai apropiat Deadline
 - optim pt sisteme real time (garantează respectarea deadline-ului)
 utilizarea CPU e sub 100%. Suma BT_i / Perioada_i = 1.

Non-p. - o dată ce planificatorul a dat CPU-ul unui proces, ăla
 rulează până se termină vs Preemptiv: POTI PUNE PAUZA

ALGORITMI PREEMPTIVI STCF/SRTF (Shortest Time-to-Completion
 First/ Shortest remaining time first) - varianta preemptivă
 a SJF - la sosirea unui job nou, dacă timpul lui rămas e
 mai mic decât timpul rămas al jobului curent, planificatorul
 întrerupe jobul curent - minimizarea optim waiting time
 mediu când joburile sosesc la momente diferite.

ROUND ROBIN (RR) - fiecare proces primește un CUAINTA de timp
 fixă, dacă nu termină în acel timp, e pus la finalul cozii READY
 și următorul e al lui; cuanta mică - răspuns rapid dar overhead
 mare de context switch; cuanta mare -> apropiere de FCFS

PRIORITY SCHEDULING - rulează procesul cu cea mai mare prioritate
 - problema STARVATION (proces cu prioritate mică nu rulează niciodată)
 - soluție AGING (prioritatea crește progresiv cu timpul de așteptare)
 MLFB (Multi Level Feedback) - mai multe cozi, cu priorități
 dif. și cuante diferite (cozi de pr. 1 = cuanta, și invers)

- un proces care consumă toată cuanta e retrogradat la coada
 de pr. mai mică; (un proces care are I/O rapid rămâne promo-
 vat la coada de prioritate mai mare) EDF (Earliest Deadline
 First) - rulează mereu jobul cu cel mai apropiat Deadline
 - optim pt sisteme real time (garantează respectarea deadline-ului)
 utilizarea CPU e sub 100%. Suma BT_i / Perioada_i = 1.

Non-p. - o dată ce planificatorul a dat CPU-ul unui proces, ăla
 rulează până se termină vs Preemptiv: POTI PUNE PAUZA

ALGORITMI PREEMPTIVI STCF/SRTF (Shortest Time-to-Completion
 First/ Shortest remaining time first) - varianta preemptivă
 a SJF - la sosirea unui job nou, dacă timpul lui rămas e
 mai mic decât timpul rămas al jobului curent, planificatorul
 întrerupe jobul curent - minimizarea optim waiting time
 mediu când joburile sosesc la momente diferite.